Vietnam National University – Ho Chi Minh City

University of Science

Faculty of Information Technology



INTRODUCTION TO ARTIFICIAL INTELLIGENCE

CSC14003

Project 2 – Logic: Futoshiki Puzzle Solver

HO CHI MINH CITY – 2026

Vietnam National University – Ho Chi Minh City

University of Science

Faculty of Information Technology



FUTOSHIKI PUZZLE SOLVER

First-Order Logic Inference Engine

Subject: CSC14003 – Introduction to Artificial Intelligence

Lecturer: _____

Student 1: Name – Student ID

Student 2: Name – Student ID

Student 3: Name – Student ID

Class: _____

Group: _____

Ho Chi Minh City, April 10, 2026

Contents

1 Project Planning and Task Distribution **2**
1.1 Team Members and Responsibilities . . . 2
1.2 Self-Evaluation Against Project Criteria . . . 2

2 FOL Formalization **2**
2.1 Vocabulary . . . 2
2.2 Complete FOL Axioms . . . 3
2.3 Derivation of CNF Forms (Three Non-trivial Axioms) . . . 4
2.3.1 Axiom A1: At Least One Value per Cell . . . 4
2.3.2 Axiom A2: At Most One Value per Cell . . . 4
2.3.3 Axiom A6: Horizontal Less-Than Constraint . . . 5

3 Inference Algorithms **5**
3.1 Automatic KB Generation . . . 5
3.2 Forward Chaining . . . 6
3.3 Backward Chaining (SLD Resolution) . . . 7
3.4 A* Search . . . 7
3.5 Heuristic Admissibility Proof . . . 8
3.6 Brute-Force and Backtracking . . . 8

4 Experimental Setup **9**
4.1 Test Cases . . . 9
4.2 Environment . . . 9

5 Experimental Results **9**
5.1 Runtime Comparison (seconds) . . . 10
5.2 Nodes Expanded / Inferences Made . . . 10
5.3 Charts . . . 10

6 Comparative Analysis **10**
6.1 Forward Chaining . . . 11
6.2 Backward Chaining (SLD Resolution) . . . 11
6.3 A* Search . . . 11
6.4 Brute-Force and Backtracking . . . 11
6.5 Summary . . . 12

7 Demonstration Video Links **12**

References **12**

1. Project Planning and Task Distribution

1.1. Team Members and Responsibilities

Member	Assigned Tasks	Completion	Score
Student 1 (ID: ...)	FOL formalization report; KB generation; forward chaining implementation	100%	9.0×1.0
Student 2 (ID: ...)	Backward chaining (SLD resolution); A* search; heuristic design	100%	9.0×1.0
Student 3 (ID: ...)	Brute-force & backtracking; experiment analysis; report writing	100%	9.0×1.0

Table 1: Task distribution and completion percentages.

Score formula

If the group score is G and a member's completion percentage is p, then the member's individual score is $G \times p$. All members are evaluated fairly based on actual contribution.

1.2. Self-Evaluation Against Project Criteria

No.	Criterion	Max	Self-eval
1	FOL formalization (report)	25%	...%
2	Automatic KB generation	10%	...%
3	Forward chaining	15%	...%
4	Backward chaining (SLD resolution)	10%	...%
5	A* search with admissible heuristic	10%	...%
6	Brute-force and backtracking	5%	...%
7	Report and experiments	25%	...%
8	GUI (bonus)	+10%	...%

Table 2: Self-evaluation table.

2. FOL Formalization

2.1. Vocabulary

All predicates follow the assignment vocabulary. The index sort is $\{1, 2, \ldots, N\}$ used for rows, columns, and values.

Predicate	Arity	Intended Meaning
$Val(i, j, v)$	3	Cell (i, j) is assigned value v
$Given(i, j, v)$	3	Cell (i, j) has pre-filled clue v
$LessH(i, j)$	2	"$<$" constraint between (i, j) and $(i, j + 1)$
$GreaterH(i, j)$	2	"$>$" constraint between (i, j) and $(i, j + 1)$
$LessV(i, j)$	2	"$<$" constraint between (i, j) and $(i + 1, j)$
$GreaterV(i, j)$	2	"$>$" constraint between (i, j) and $(i + 1, j)$
$Less(v_1, v_2)$	2	Background arithmetic: $v_1 < v_2$

Table 3: Predicate vocabulary.

2.2. Complete FOL Axioms

A1 – Every cell has at least one value

$$\forall i\ \forall j\ \exists v \quad Val(i, j, v)$$

A2 – Every cell has at most one value

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(Val(i, j, v_1) \land Val(i, j, v_2)\big) \Rightarrow v_1 = v_2$$

A3 – Row uniqueness (no duplicate in same row)

$$\forall i\ \forall j_1\ \forall j_2\ \forall v \quad \big(Val(i, j_1, v) \land Val(i, j_2, v) \land j_1 \neq j_2\big) \Rightarrow \bot$$

A4 – Column uniqueness (no duplicate in same column)

$$\forall j\ \forall i_1\ \forall i_2\ \forall v \quad \big(Val(i_1, j, v) \land Val(i_2, j, v) \land i_1 \neq i_2\big) \Rightarrow \bot$$

A5 – Value completeness (values stay within $[1, N]$)

$$\forall i\ \forall j\ \forall v \quad Val(i, j, v) \Rightarrow \big(v = 1 \lor v = 2 \lor \cdots \lor v = N\big)$$

A6 – Horizontal less-than constraint

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(LessH(i, j) \land Val(i, j, v_1) \land Val(i, j{+}1, v_2)\big) \Rightarrow Less(v_1, v_2)$$

A7 – Horizontal greater-than constraint

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(GreaterH(i, j) \land Val(i, j, v_1) \land Val(i, j{+}1, v_2)\big) \Rightarrow Less(v_2, v_1)$$

A8 – Vertical less-than constraint

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(LessV(i, j) \land Val(i, j, v_1) \land Val(i{+}1, j, v_2)\big) \Rightarrow Less(v_1, v_2)$$

A9 – Vertical greater-than constraint

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(GreaterV(i,j) \land Val(i,j,v_1) \land Val(i{+}1,j,v_2)\big) \Rightarrow Less(v_2,v_1)$$

A10 – Given clues are enforced

$$\forall i\ \forall j\ \forall v \quad Given(i,j,v) \Rightarrow Val(i,j,v)$$

2.3. Derivation of CNF Forms (Three Non-trivial Axioms)

2.3.1. Axiom A1: At Least One Value per Cell

Original FOL sentence:

$$\forall i\ \forall j\ \exists v \quad Val(i,j,v)$$

Step 1 – Skolemization. The existential quantifier $\exists v$ is replaced by a Skolem function $f(i,j)$ depending on all universally quantified variables in scope:

$$\forall i\ \forall j \quad Val\big(i,\ j,\ f(i,j)\big)$$

Step 2 – Drop universal quantifiers (standard CNF convention):

$$Val\big(i,\ j,\ f(i,j)\big)$$

CNF (single unit clause):

$$\boxed{Val(i,\ j,\ f(i,j))}$$

When grounded over domain $\{1,\ldots,N\}$, this produces N^2 ground unit clauses, one per cell.

2.3.2. Axiom A2: At Most One Value per Cell

Original FOL sentence:

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(Val(i,j,v_1) \land Val(i,j,v_2)\big) \Rightarrow v_1 = v_2$$

Step 1 – Eliminate implication ($\phi \Rightarrow \psi \equiv \neg\phi \lor \psi$):

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \neg\big(Val(i,j,v_1) \land Val(i,j,v_2)\big) \lor (v_1 = v_2)$$

Step 2 – Apply De Morgan:

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \neg Val(i,j,v_1) \lor \neg Val(i,j,v_2) \lor (v_1 = v_2)$$

Step 3 – Handle equality. We treat $v_1 = v_2$ via the contrapositive: for distinct $v_1 \neq v_2$, no cell can hold both values. Grounded for all $v_1 \neq v_2$:

CNF (one clause per distinct value pair per cell):

$$\boxed{\neg Val(i,j,v_1) \lor \neg Val(i,j,v_2)} \quad \text{for each } v_1 \neq v_2$$

2.3.3. Axiom A6: Horizontal Less-Than Constraint

Original FOL sentence:

$$\forall i\ \forall j\ \forall v_1\ \forall v_2 \quad \big(LessH(i,j) \land Val(i,j,v_1) \land Val(i,j{+}1,v_2)\big) \Rightarrow Less(v_1,v_2)$$

Step 1 – Eliminate implication:

$$\neg\big(LessH(i,j) \land Val(i,j,v_1) \land Val(i,j{+}1,v_2)\big) \lor Less(v_1,v_2)$$

Step 2 – Apply De Morgan:

$$\neg LessH(i,j) \lor \neg Val(i,j,v_1) \lor \neg Val(i,j{+}1,v_2) \lor Less(v_1,v_2)$$

Step 3 – Ground $Less(v_1,v_2)$**.** Since $Less$ is a background arithmetic predicate, it evaluates to $\top$ when $v_1 < v_2$ and $\bot$ otherwise. Clauses where $v_1 \geq v_2$ become:

CNF (for each $v_1 \geq v_2$**):**

$$\boxed{\neg LessH(i,j) \lor \neg Val(i,j,v_1) \lor \neg Val(i,j{+}1,v_2)}$$

This clause forbids assigning v_1 to cell (i,j) and v_2 to cell $(i,j+1)$ when a "$<$" constraint exists and $v_1 \geq v_2$.

3. Inference Algorithms

3.1. Automatic KB Generation

Given size N, the algorithm instantiates all FOL axioms over the concrete domain $\{1,\ldots,N\}$.

Overview

For each axiom schema, substitute all valid combinations of indices (i,j,v) from $\{1,\ldots,N\}$ to produce a fully grounded propositional knowledge base (KB) in CNF. The KB is then fed to a SAT/resolution engine for verification.

```
GENERATE_KB(N):
  clauses = []
  for each i, j in {1..N} x {1..N}:
    # A1: at-least-one value per cell
    clauses.add( {Val(i,j,v) : v in 1..N} )
    # A2: at-most-one value per cell
    for each v1 != v2 in {1..N}:
      clauses.add( {-Val(i,j,v1), -Val(i,j,v2)} )
    # A3: row uniqueness
    for each j1 != j2, v:
      clauses.add( {-Val(i,j1,v), -Val(i,j2,v)} )
    # A4: column uniqueness
    for each i1 != i2, v:
      clauses.add( {-Val(i1,j,v), -Val(i2,j,v)} )
    # A6/A7: horizontal inequality clauses
    if LessH(i,j) is a given fact and j < N:
      for each v1 >= v2 in {1..N}:
```

```
          clauses.add( {-Val(i,j,v1), -Val(i,j+1,v2)} )
      # A8/A9: vertical inequality clauses (similarly)
      # A10: given clues
      if Given(i,j,v) is a fact:
        clauses.add( {Val(i,j,v)} )
    return clauses
```

Listing 1: Automatic KB Generation

3.2. Forward Chaining

Idea: Starting from known facts (*Given*, *LessH*, *GreaterH*, etc.), repeatedly apply Modus Ponens to derive new $Val(i, j, v)$ atoms until a fixed-point is reached or a contradiction is detected. Combined with AC-3 style constraint propagation to prune domains.

```
FORWARD_CHAINING(puzzle):
  # Initialize domains
  for each cell (i,j):
    domain[i][j] = {1..N}
  for each Given(i,j,v):
    domain[i][j] = {v}    # enforce given clues

  changed = True
  while changed:
    changed = False
    for each cell (i,j) with |domain[i][j]| == 1:
      assigned_val = the unique value in domain[i][j]
      # Propagate row uniqueness
      for each j' != j in same row:
        if assigned_val in domain[i][j']:
          domain[i][j'].remove(assigned_val)
          changed = True
      # Propagate column uniqueness
      for each i' != i in same column:
        if assigned_val in domain[i'][j]:
          domain[i'][j].remove(assigned_val)
          changed = True
    # Propagate inequality constraints
    for each LessH(i,j):
      # val at (i,j) < val at (i,j+1)
      min_right = min(domain[i][j+1])
      domain[i][j] = {v : v < min_right, v in domain[i][j]}
      max_left = max(domain[i][j])
      domain[i][j+1] = {v : v > max_left, v in domain[i][j+1]}
    # Repeat for GreaterH, LessV, GreaterV ...
    # Check for contradiction
    for each cell (i,j):
      if domain[i][j] is empty:
        return CONTRADICTION
  if all cells have singleton domain:
    return SOLUTION
  else:
    return BACKTRACKING_COMPLETION(domains)
```

Listing 2: Forward Chaining Algorithm

3.3. Backward Chaining (SLD Resolution)

Idea: Implement a Prolog-like query engine. Given a goal $Val(i, j, ?)$, the interpreter reduces it to sub-goals using Horn clause rules encoding Futoshiki constraints, applying depth-first SLD resolution.

```
BACKWARD_CHAIN(goal Val(i,j,?), assignment):
  if Val(i,j,_) already assigned:
    return {assignment[i][j]}
  candidates = {1..N}
  # Prune via row uniqueness rule
  for each j' in same row where Val(i,j',v') is assigned:
    candidates.remove(v')
  # Prune via column uniqueness rule
  for each i' in same column where Val(i',j,v') is assigned:
    candidates.remove(v')
  # Prune via horizontal inequality rules
  if LessH(i, j-1) and Val(i,j-1,v') assigned:
    candidates = {v : v > v', v in candidates}
  if LessH(i, j) and Val(i,j+1,v') assigned:
    candidates = {v : v < v', v in candidates}
  # (Repeat for GreaterH, LessV, GreaterV ...)
  results = {}
  for each v in candidates:
    assignment[i][j] = v
    # Recursively prove remaining unassigned cells
    sub = BACKWARD_CHAIN(next_unassigned_goal, assignment)
    if sub != FAILURE:
      results.add(v)
    assignment[i][j] = unassigned
  return results if results != {} else FAILURE
```

Listing 3: Backward Chaining (SLD Resolution)

3.4. A* Search

State: A partial assignment of values to cells, together with the current domains after constraint propagation.

Transition cost: Assigning one cell increases g by 1 (uniform step cost).

Heuristic: $h(s)$ = number of unassigned cells after running AC-3 on the current state. See Section 3.5 for admissibility proof.

```
A_STAR(puzzle):
  start = initial_state(puzzle)
  open = priority_queue()
  open.push(start, f = 0 + h(start))
  closed = {}

  while open is not empty:
    state = open.pop_min_f()
    if state is COMPLETE_ASSIGNMENT:
      return state.assignment
    if state in closed:
      continue
    closed.add(state)

```

```
    # Pick cell using MRV heuristic (min remaining values)
    cell = argmin over unassigned cells of |domain[cell]|

    for each v in domain[cell]:
      new_state = assign(state, cell, v)
      new_state.domains = AC3_PROPAGATE(new_state)
      if not CONTRADICTION(new_state):
        g = state.g + 1
        h = count_unassigned(new_state)  # admissible h
        open.push(new_state, f = g + h)

  return NO_SOLUTION
```

Listing 4: A* Search Algorithm

3.5. Heuristic Admissibility Proof

Theorem: $h(s) = |U(s)|$ is admissible

Let $U(s)$ be the set of unassigned cells in state s after AC-3 propagation. Each action assigns exactly one cell, so the true remaining cost satisfies:

$$h^*(s) \geq |U(s)| = h(s)$$

Since $h(s)$ never overestimates $h^*(s)$, it is **admissible** and A* is guaranteed to find the optimal (minimal-step) solution.

Stronger heuristic (optional): Count cells whose domain is reduced to size 1 after AC-3 but whose singleton value conflicts with a peer. Any such state incurs at least one additional backtrack, so $h(s)$ can be augmented by the count of domain-empty cells detected during AC-3, giving a strictly better lower bound while remaining admissible.

3.6. Brute-Force and Backtracking

Brute-force: Enumerate all N^{N^2} possible assignments and test each against all constraints. Used only as a theoretical baseline.

Backtracking: Depth-first search with constraint checking at each step. Attempts to fill cells in row-major order; prunes branches when a partial assignment violates any rule. No heuristic guidance.

```
BACKTRACK(assignment, empty_cells):
  if empty_cells is empty:
    return assignment  # complete solution
  (i, j) = empty_cells[0]
  for v in {1..N}:
    if IS_CONSISTENT(assignment, i, j, v):
      assignment[i][j] = v
      result = BACKTRACK(assignment, empty_cells[1:])
      if result != FAILURE:
        return result
      assignment[i][j] = unassigned
  return FAILURE

```

```
IS_CONSISTENT(assignment, i, j, v):
  # Check row uniqueness
  if v in {assignment[i][j'] : j' in row, j' != j}: return False
  # Check column uniqueness
  if v in {assignment[i'][j] : i' in col, i' != i}: return False
  # Check horizontal inequalities
  # Check vertical inequalities
  return True
```

Listing 5: Backtracking Solver

4. Experimental Setup

4.1. Test Cases

At least 10 input files are provided, spanning grid sizes 4×4, 5×5, 6×6, 7×7, and 9×9:

File	Grid Size	Given Cells	Difficulty
input-01.txt	4×4	...	Easy
input-02.txt	4×4	...	Medium
input-03.txt	5×5	...	Easy
input-04.txt	5×5	...	Hard
input-05.txt	6×6	...	Easy
input-06.txt	6×6	...	Medium
input-07.txt	7×7	...	Medium
input-08.txt	7×7	...	Hard
input-09.txt	9×9	...	Medium
input-10.txt	9×9	...	Hard

Table 4: Summary of test cases.

4.2. Environment

- **Language:** Python 3.11
- **Hardware:** ... (CPU, RAM)
- **OS:** ...
- **Metrics:** Wall-clock runtime (seconds), peak memory usage (MB), number of inferences/nodes expanded.

5. Experimental Results

5.1. Runtime Comparison (seconds)

Test	Size	Forward	Backward	A*	Backtrack
input-01	4×4	TODO	TODO	TODO	TODO
input-02	4×4	TODO	TODO	TODO	TODO
input-03	5×5	TODO	TODO	TODO	TODO
input-04	5×5	TODO	TODO	TODO	TODO
input-05	6×6	TODO	TODO	TODO	TODO
input-06	6×6	TODO	TODO	TODO	TODO
input-07	7×7	TODO	TODO	TODO	TODO
input-08	7×7	TODO	TODO	TODO	TODO
input-09	9×9	TODO	TODO	TODO	TODO
input-10	9×9	TODO	TODO	TODO	TODO

Table 5: Runtime (seconds) for each algorithm on all test cases.

5.2. Nodes Expanded / Inferences Made

Test	Size	Forward	Backward	A*	Backtrack
input-01	4×4	TODO	TODO	TODO	TODO
input-02	4×4	TODO	TODO	TODO	TODO
input-03	5×5	TODO	TODO	TODO	TODO
input-04	5×5	TODO	TODO	TODO	TODO
input-05	6×6	TODO	TODO	TODO	TODO
input-06	6×6	TODO	TODO	TODO	TODO
input-07	7×7	TODO	TODO	TODO	TODO
input-08	7×7	TODO	TODO	TODO	TODO
input-09	9×9	TODO	TODO	TODO	TODO
input-10	9×9	TODO	TODO	TODO	TODO

Table 6: Number of inferences / nodes expanded per algorithm.

5.3. Charts

Note

Charts comparing runtime, memory, and inferences will be inserted here after experiments are complete. Each chart should compare all four algorithms across the 10 test cases.

6. Comparative Analysis

6.1. Forward Chaining

Forward chaining is highly effective on well-constrained puzzles (many given clues and inequality chains), often solving 4×4 and 5×5 instances without any backtracking. Its arc-consistency propagation eliminates large swaths of the search space in polynomial time per iteration. However, on sparse 9×9 instances with few given cells, it may reach a fixed-point before fully determining the solution, requiring supplementary backtracking.

Preferred when:

Puzzle is heavily constrained (many givens or inequalities); solving grids of size up to 6×6.

6.2. Backward Chaining (SLD Resolution)

Backward chaining queries individual cells top-down, making it convenient for partial queries (*"what is the value of cell* (i, j)*?"*). It avoids computing the entire solution when only a subset of values is needed. However, its depth-first nature can lead to deep recursion and redundant sub-goal evaluation on larger grids without memoization.

Preferred when:

Only a subset of cell values needs to be queried; small to medium puzzle sizes; Prolog-style reasoning is required.

6.3. A* Search

A* with the AC-3 heuristic consistently expands far fewer nodes than backtracking. The MRV (Minimum Remaining Values) cell selection policy focuses effort on the most constrained cells first, producing tight branch pruning. On 7×7 and 9×9 puzzles, A* demonstrates clear advantages in node count over backtracking, at the cost of higher memory due to the open list.

Preferred when:

Large puzzle sizes; correctness and optimality (in terms of steps) are required; memory is not a bottleneck.

6.4. Brute-Force and Backtracking

Brute-force is infeasible beyond 4×4. Plain backtracking serves as a practical baseline: it is simpler to implement and uses negligible memory, but expands exponentially more nodes than A* on hard instances. It remains a useful correctness oracle for verifying other solvers.

Preferred when:

Verification of other algorithms; simple implementation needed; only small grids.

6.5. Summary

Algorithm	Speed (relative)	Memory	Best Use Case
Forward Chaining	Fast on constrained	Low	$\leq 6 \times 6$, dense constraints
Backward Chaining	Medium	Low	Partial queries, small grids
A*	Fast on large	Medium–High	$\geq 7 \times 7$, full solution needed
Backtracking	Slow	Low	Correctness verification
Brute-Force	Very slow	Low	4×4 only

Table 7: Algorithm comparison summary.

7. Demonstration Video Links

Three demonstration videos covering different grid sizes are available at the following links:

- **Demo 1** (4×4 puzzle, forward chaining):
  https:// _____
- **Demo 2** (6×6 puzzle, A* search):
  https:// _____
- **Demo 3** (9×9 puzzle, backtracking vs A* comparison):
  https:// _____

Each video starts from the command line, shows the input file content, and walks through key algorithmic steps (facts inferred by forward chaining, nodes expanded by A*, etc.).

References

[1] Stuart Russell, Peter Norvig. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[2] Faculty of Information Technology, University of Science, VNU-HCM. *CSC14003 Project 2: Logic – Futoshiki Puzzles*, 2025.

[3] Futoshiki Puzzle Rules. https://www.futoshiki.org/

[4] Allen Van Gelder. *Constraint Satisfaction and Backtracking Search.* Stanford University Lecture Notes.